Memo No.
Date / /

problem set 8 answers

problem 1. Algorithm to compute rev(G) in O(V+E)

Idea: reverse every edge.

Pseudo-code:

```
rev(G):
  create empty graph G-rev
  for each vertex v in G:
      add v to G-rev
  for each edge (u->v) in G:
      add edge (v->u) to G-rev
  return G-rev
```

Time complexity:

Visit all vertices -> O(V)

Visit all edges -> O(E)

so total: O(V+E)

Mo Tu We Th Fr Sa Su

Memo No.

Date / /

2. prove SCC(G) is acyclic

Explanitation:

SCC(G) = each strongly connected component becomes one node

There is an edge between components if original graph has it

proof idea:

Assume SCC(G) has a cycle:

$C1 \to C2 \to C3 \to \cdots \to C1$

This means:

from C1 we can reach C2

from C2 we can reach C3

...

from last we can reah C1

so all these components are mutually reachable.

Therefore they should be one single SCC, not different ones.

contradiction.

so:

SCC(G) is acyclic (a DAG)

Memo No.
Date / /

3. prove scc (rev(G)) = rev (scc(G))

Explanation:

Reversing graph filps all edges

SCC groups depend on mutual reachabillity

key idea:

if u and v are in same SCC in G:

$u \to v$ and $v \to u$ exist

After reversal: $v \to u$ and $u \to v$ still exist

So SCCs do not change

only edges between SCCs reverse.

so: scc (rev (G)) = rev (scc (G))

Memo No.
Date / /

4. Reachability relation

Statement:

u can reach v in G $\Leftrightarrow$ S(u) can reach S(v) in SCC(G)

proof:

If $u \to v$ exists:

u is in S(u)

v is in S(v)

so there is a path:

$S(u) \to S(v)$

If: $S(u) \to S(v)$

Then: there exists path between components

since $u \in S(u)$, $v \in S(v)$

so: $u \to v$ in original graph

Memo No.
Date / /

Problem 2 1. condition for Euler tour:
Graph has Euler tour iff:
in-degree(v) = out-degree(v)
for every vertex v.
Explanation: Every time we enter a vertex, we must leave it.
. so number of incoming edges = outgoing edges
2. Algorithm to find Euler tour (O(E))
Algorithm idea: Hierholzer's Algorithm
Pseudo-code:

```
Euler(G):
    start from any vertex V
    stack = [V]
    path = [ ]
    while stack not empty:
        current = top of stack
        if current has unused edges:
            choose edge (current → u)
            remove edge
            push u to stack
        else: pop current
              add to path
    return path
```

Memo No.
Date / /

Explanation:
Build small cycles
Merge them into one big cycle
Time complexity: $O(E)$

Problem 3 (Topological Sort)
Graph: $A \to B$
$A \to C$
$B \to C$
$B \to D$
$C \to E$
$D \to E$
$D \to F$
$G \to F$
$G \to E$

Memo No.
Date / /

start from A

step-by-step:

A → B → D → F → C → E → G

one vallid ordering:

A, B, D, F, C, E, G

Another valid ordering:

A, B, C, D, E, F, G

2. Start from another node (example G)

order:

G, A, B, C, D, E, F

Another:

G, A, B, D, C, F, E

Explanation:

Topological sort rule:

If u → v, then u must appear before v